

SBA301 – SLOT 05

DỰ ÁN TỔNG HỢP REACTJS VỚI INTELLIJ IDEA

EVENTHUB – CAMPUS EVENT EXPLORER

Hướng dẫn chi tiết từng bước • Mức độ trung bình • Frontend Integration Project

Mục tiêu cốt lõi Sinh viên tự xây một ứng dụng React chạy được và giải thích được luồng: static data → map() → reusable component → props → event → useState → re-render → Modal; đồng thời biết kiểm chứng bằng terminal, browser console, manual tests và Git.

Phạm vi Dự án này là bài củng cố/transfer từ Slot 05. Không dùng Router, Axios, REST, Spring Boot, database hoặc global state library. React Router sẽ học ở Slot 06.

Thông tin	Giá trị
Môn học	SBA301 – Integrate Single Page Application with Spring Boot
Baseline	SBA301-40 v2
Củng cố	Slot 05 – Integrated React Lab 01
IDE	IntelliJ IDEA
Mức độ	Trung bình
Sản phẩm	EventHub – Campus Event Explorer
Đầu ra	Ứng dụng chạy được + source code + test evidence + Git checkpoint + giải thích data flow

1. Cách sử dụng tài liệu

Tài liệu được viết theo mô hình zero-assumption: mỗi bước đều có thao tác, mục đích, kết quả mong đợi, cách kiểm tra và cách xử lý khi thất bại. Không nên copy toàn bộ source rồi chạy một lần.

Ký hiệu	Ý nghĩa
THAO TÁC	Việc sinh viên phải thực hiện trong IntelliJ/Terminal/Browser.
WHY	Tại sao bước này cần thiết.
EXPECTED	Kết quả mong đợi sau bước.
CHECKPOINT	Điều kiện PASS/FAIL trước khi đi tiếp.
IF FAILED	Chẩn đoán tối thiểu, không sửa nhiều thứ cùng lúc.
EVIDENCE	Bằng chứng cần lưu để chứng minh kết quả.

Nguyên tắc thực hành Một thay đổi nhỏ → Run → Verify → nếu lỗi thì đọc evidence → sửa đúng một nguyên nhân → Run lại.
Không refactor trong lúc chưa biết lỗi ở đâu.

2. Thông tin dự án và mục tiêu

EventHub là một SPA frontend mô phỏng cổng khám phá sự kiện trong trường đại học. Dữ liệu sự kiện được lưu tạm trong module JavaScript phía frontend. Người dùng có thể xem danh sách, tìm kiếm/lọc, mở chi tiết và đánh dấu quan tâm ở mức local state.

Dự án được chọn khác domain Orchid để kiểm tra khả năng chuyển giao kiến thức: sinh viên phải áp dụng cùng mental model của Slot 05 vào một collection screen mới thay vì chỉ sao chép Lab Orchid.

Mục tiêu kỹ thuật	Evidence bắt buộc
Component Architecture	Component tree + trách nhiệm từng component
React-Bootstrap	Navbar, Container/Row/Col, Card, Form, Modal
Props	Một Event object truyền vào EventCard
Collection rendering	events.map(...) + stable key
State	search/category/featured filter và show detail
Event-driven UI	Button/Form event làm state thay đổi và UI re-render
Debugging	Terminal + browser console không có blocking error
Version control	Git commit tại working checkpoint

3. Chức năng của ứng dụng

- Hiển thị Navbar và phần giới thiệu EventHub.

- Hiển thị tối thiểu 8 sự kiện từ static data module.
- Mỗi sự kiện hiển thị bằng cùng một reusable EventCard.
- Render danh sách bằng Array.map() và key ổn định theo event.id.
- Tìm kiếm theo tên sự kiện.
- Lọc theo category.
- Bật/tắt chế độ Featured only.
- Hiển thị số kết quả hiện tại như derived value.
- Mở/đóng chi tiết sự kiện bằng React-Bootstrap Modal và local useState.
- Hiển thị empty state khi bộ lọc không còn kết quả.
- Responsive grid trên màn hình lớn/nhỏ.
- Không dùng Router; các Nav.Link chỉ là UI placeholder trong Slot 05.

4. Công nghệ và giới hạn

Công nghệ	Vai trò trong dự án	Phạm vi
ReactJS	Component, props, state, rendering	Core
Vite	Scaffold + dev server + build	Core
React-Bootstrap	Navbar, Card, Grid, Form, Modal	Core
JavaScript	Array, object, filter(), map(), string matching	Core
IntelliJ IDEA	IDE, terminal, npm run configuration	Core
Git	Working checkpoint	Core
Context	Không bắt buộc; chỉ phân tích quyết định thiết kế	Decision evidence
React Router	Navigation thật giữa views	Learn Later – Slot 06
Axios/REST/Spring Boot/DB	Backend integration/persistence	Learn Later

Version note Nếu môi trường môn học vẫn dùng Node 21, hãy dùng phiên bản Vite/create-vite đã được giảng viên phê duyệt (ví dụ nhánh Vite 5) thay vì tự động chuyển sang @latest. Nếu lớp đã nâng Node theo hướng dẫn mới, dùng command do giảng viên chỉ định. Mục tiêu là môi trường nhất quán, không phải chạy phiên bản mới nhất bằng mọi giá.

5. Kiến trúc và luồng dữ liệu

```

main.jsx
↓
<App />
├─ AppNavbar
├─ HeroSection
├─ EventList
│   └─ filter state (search/category/featured)
│       └─ derived filteredEvents
│           └─ EventCard × N

```

```

|
|   └─ event prop
|
|   └─ local showDetail state → Modal
└─ AppFooter

```

Luồng collection:

```

events.js
  ↓ import
EventList
  ↓ filter()
filteredEvents
  ↓ map()
<EventCard event={event} />
  ↓ props
JSX / Card

```

Luồng tương tác chi tiết:

```

View Detail click
  ↓
onClick handler
  ↓
setShowDetail(true)
  ↓
state changes
  ↓
EventCard renders again
  ↓
Modal visible

```

6. Cấu trúc thư mục mục tiêu

```

eventhub-campus-explorer/
├─ public/
│   └─ images/
│       └─ event-placeholder.svg
├─ src/
│   ├── components/
│   │   ├── AppNavbar.jsx
│   │   ├── HeroSection.jsx
│   │   ├── EventList.jsx
│   │   ├── EventCard.jsx
│   │   └─ AppFooter.jsx
│   ├── data/
│   │   └─ events.js
│   ├── App.jsx
│   ├── App.css
│   └─ main.jsx
├─ package.json
└─ vite.config.js

```

Lưu ý Tên file có thể khác một chút tùy template Vite. Không chỉnh cấu trúc chỉ để giống hình; ưu tiên hiểu vai trò và import path thực tế.

7. Tóm tắt 30 bước thực hiện

#	Bước	Điều kiện hoàn thành
1	Mở IntelliJ và tạo workspace	IDE mở đúng thư mục
2	Kiểm tra Node/npm/Git	Các lệnh trả version
3	Scaffold Vite React project	package.json xuất hiện
4	npm install và baseline run	Trang Vite chạy
5	Cấu hình Node interpreter trong IntelliJ	IDE nhận Node/npm
6	Cài React-Bootstrap	Dependencies có package
7	Import Bootstrap CSS	Styling hoạt động
8	Tạo npm Run Configuration	Run bằng nút IntelliJ
9	Dọn starter code có kiểm soát	App tối giản vẫn chạy
10	Vẽ component tree	Giải thích được trách nhiệm
11	Tạo local SVG asset	Browser đọc được ảnh
12	Tạo events.js	Có 8 records hợp lệ
13	Tạo AppNavbar	Navbar hiển thị
14	Tạo HeroSection	Heading/description hiển thị
15	Tạo EventCard baseline	Một card hoạt động
16	Truyền event bằng Props	Card hiển thị record khác nhau
17	Tạo EventList + map()	Render N cards
18	Dùng stable key	Không còn key warning
19	Responsive grid	Cards thích nghi viewport
20	Thêm search state	Search lọc đúng
21	Thêm category state	Category lọc đúng
22	Thêm featured-only state	Toggle lọc đúng
23	Derived result count + empty state	Số lượng và empty state đúng
24	Thêm local Modal state	View Detail mở/đóng
25	Kiểm tra Props vs State	Giải thích ownership

#	Bước	Điều kiện hoàn thành
26	Bug injection + debugging	Tìm được lỗi có evidence
27	Manual test matrix	Test cases PASS/FAIL
28	Git checkpoint	git log -1 có commit
29	npm run build + preview	Production build chạy
30	Final verification + evidence package	Đủ artifact để nộp/giải thích

8. Hướng dẫn chi tiết từng bước

Bước 1. Mở IntelliJ IDEA và tạo workspace

THAO TÁC: Mở IntelliJ IDEA → File → Open → chọn thư mục SBA301/labs hoặc một workspace cá nhân.

WHY: Đảm bảo project được tạo trong vị trí có thể tìm lại, commit và lưu evidence.

EXPECTED: Project view hiển thị đúng workspace và Terminal của IntelliJ mở được.

CHECKPOINT — ☐ IntelliJ mở bình thường | ☐ Terminal mở được | ☐ Đường dẫn workspace đúng

Bước 2. Kiểm tra Node, npm và Git

THAO TÁC: Mở Terminal trong IntelliJ và chạy các lệnh sau.

```
node -v
npm -v
git --version
```

EXPECTED: Mỗi lệnh trả về version; không có “not recognized” / “command not found”.

IF FAILED: Dừng việc tạo project. Sửa PATH/Node/Git trước; không đổ lỗi cho React source code khi executable chưa chạy.

CHECKPOINT — ☐ node PASS | ☐ npm PASS | ☐ git PASS

Bước 3. Scaffold React project bằng Vite

THAO TÁC: Trong Terminal, dùng command tương thích với môi trường lớp. Với Node 21/course pin, ví dụ:

```
npm create vite@5 eventhub-campus-explorer -- --template react
```

Nếu giảng viên đã cập nhật môi trường và yêu cầu Vite mới hơn, thay command trên bằng command chính thức của lớp.

THAO TÁC: Đi vào project:

```
cd eventhub-campus-explorer
```

EXPECTED: Terminal path kết thúc bằng eventhub-campus-explorer và có package.json.

CHECKPOINT — ☐ package.json có mặt | ☐ src/ có mặt | ☐ Terminal đang ở project root

Bước 4. Cài dependencies và chạy baseline

THAO TÁC: Chạy:

```
npm install
npm run dev
```

EXPECTED: Terminal hiển thị Local URL; browser mở được starter page.

VERIFY: Mở URL do Vite in ra. Nhấn F12 → Console; không có blocking error.

IF FAILED: Đọc lỗi đầu tiên. Kiểm tra working directory, package.json và npm install trước khi chỉnh source.

CHECKPOINT — ☐ Dev server chạy | ☐ Browser mở | ☐ Console không blocking error

Bước 5. Kiểm tra Node interpreter trong IntelliJ

THAO TÁC: Mở Settings/Preferences và tìm “Node.js” bằng ô Search. Xác nhận Node interpreter trả đúng executable đang dùng trong Terminal.

Tên đường dẫn menu có thể thay đổi theo phiên bản IntelliJ/plugin JavaScript; ô Search trong Settings là cách an toàn nhất để tìm cấu hình Node.js.

EXPECTED: IntelliJ nhận Node và npm; package.json có gutter/run actions hoặc npm scripts nhận diện được.

CHECKPOINT — ☐ Node interpreter hợp lệ | ☐ npm scripts được IDE nhận diện

Bước 6. Cài React-Bootstrap và Bootstrap

THAO TÁC: Dừng dev server nếu cần bằng Ctrl+C, sau đó chạy:

```
npm install react-bootstrap bootstrap
```

VERIFY: Mở package.json và kiểm tra dependencies có react-bootstrap và bootstrap.

CHECKPOINT — ☐ Installation không blocking error | ☐ package.json có dependencies

Bước 7. Import Bootstrap CSS

THAO TÁC: Mở src/main.jsx và thêm import CSS trước custom CSS.

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

WHY: React-Bootstrap cung cấp React components; Bootstrap CSS cung cấp style tương ứng.

VERIFY: Chạy lại dev server; project vẫn compile.

CHECKPOINT — ☐ CSS import đúng | ☐ Dev server vẫn PASS

Bước 8. Tạo npm Run Configuration trong IntelliJ

THAO TÁC: Run → Edit Configurations → Add New Configuration → npm. Chọn package.json của project, Command = run, Script = dev. Đặt tên “EventHub Dev”.

EXPECTED: Có thể nhấn Run/Debug từ IntelliJ thay vì gõ npm run dev mỗi lần.

CHECKPOINT — ☐ Configuration lưu được | ☐ Run mở Vite server

Bước 9. Dọn starter code có kiểm soát

THAO TÁC: Thay App.jsx bằng baseline tối giản; không xóa toàn bộ project.

```
function App() {
  return <h1>EventHub</h1>;
}

export default App;
```

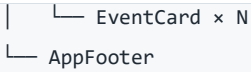
VERIFY: Browser hiển thị EventHub. Đây là working baseline sau cleanup.

CHECKPOINT — ☐ App tối giản hiển thị | ☐ Không compile error

Bước 10. Thiết kế component tree trước khi code

THAO TÁC: Vẽ component tree sau vào notes hoặc giấy; ghi trách nhiệm từng node.

```
App
├── AppNavbar
├── HeroSection
└── EventList
```



Component	Trách nhiệm	State/Props chính
App	Compose application	Không cần state core
AppNavbar	Navigation UI placeholder	Không state
HeroSection	Giới thiệu app	Không state
EventList	Collection + filters	searchText, category, featuredOnly
EventCard	Hiển thị 1 event + detail	event prop; showDetail state
AppFooter	Footer	Không state

CHECKPOINT — ☐ Tree có root | ☐ Mỗi component có trách nhiệm rõ | ☐ Không tách mỗi div thành component

Bước 11. Tạo local SVG asset không phụ thuộc Internet

THAO TÁC: Tạo public/images/event-placeholder.svg và dán nội dung sau:

```
<svg xmlns="http://www.w3.org/2000/svg" width="800" height="450" viewBox="0 0 800 450">
  <rect width="800" height="450" fill="#eaf2f8"/>
  <circle cx="400" cy="180" r="70" fill="#f28c28"/>
  <text x="400" y="310" text-anchor="middle" font-family="Arial" font-size="42" fill="#163a5f">EventHub</text>
</svg>
```

VERIFY: Mở trực tiếp http://localhost:<port>/images/event-placeholder.svg trong browser.

CHECKPOINT — ☐ SVG load được | ☐ Không 404

Bước 12. Tạo static data module

THAO TÁC: Tạo src/data/events.js.

```
export const events = [
  {
    id: 1,
    title: "AI & Software Engineering Workshop",
    category: "Technology",
    date: "2026-09-20",
    location: "Innovation Hall A",
    featured: true,
    seats: 120,
    description: "Hands-on workshop about practical AI-assisted software engineering.",
    image: "/images/event-placeholder.svg"
  },
  {
    id: 2,
    title: "Campus Career Day",
    category: "Career",
    date: "2026-09-22",
    location: "Main Auditorium",
    featured: true,
```

```
    seats: 300,
    description: "Meet employers and explore internship and graduate opportunities.",
    image: "/images/event-placeholder.svg"
  },
  {
    id: 3,
    title: "Green Campus Volunteer Morning",
    category: "Community",
    date: "2026-09-24",
    location: "Campus Garden",
    featured: false,
    seats: 80,
    description: "Volunteer activity focused on campus cleaning and tree care.",
    image: "/images/event-placeholder.svg"
  },
  {
    id: 4,
    title: "Interfaculty Football Match",
    category: "Sports",
    date: "2026-09-26",
    location: "University Stadium",
    featured: false,
    seats: 500,
    description: "Friendly football tournament between faculties.",
    image: "/images/event-placeholder.svg"
  },
  {
    id: 5,
    title: "Startup Idea Pitch Night",
    category: "Entrepreneurship",
    date: "2026-09-28",
    location: "Demo Room 2",
    featured: true,
    seats: 100,
    description: "Student teams pitch early startup ideas and receive feedback.",
    image: "/images/event-placeholder.svg"
  },
  {
    id: 6,
    title: "English Speaking Club",
    category: "Academic",
    date: "2026-09-29",
    location: "Room B205",
    featured: false,
    seats: 40,
    description: "Weekly speaking practice through games and discussion topics.",
    image: "/images/event-placeholder.svg"
```

```

    },
    {
      id: 7,
      title: "Cybersecurity Capture the Flag",
      category: "Technology",
      date: "2026-10-02",
      location: "Lab C301",
      featured: true,
      seats: 60,
      description: "Beginner-friendly cybersecurity challenge in teams.",
      image: "/images/event-placeholder.svg"
    },
    {
      id: 8,
      title: "Student Music Evening",
      category: "Culture",
      date: "2026-10-04",
      location: "Outdoor Stage",
      featured: false,
      seats: 250,
      description: "Live performances by student bands and clubs.",
      image: "/images/event-placeholder.svg"
    }
  ]
};

```

WHY: events.js là static frontend data module, không phải database.

CHECKPOINT — ☐ 8 records | ☐ id unique | ☐ Các field nhất quán | ☐ image path hợp lệ

Bước 13. Tạo AppNavbar

THAO TẮC: Tạo src/components/AppNavbar.jsx.

```

import { Container, Nav, Navbar } from "react-bootstrap";

function AppNavbar() {
  return (
    <Navbar bg="dark" data-bs-theme="dark" expand="lg">
      <Container>
        <Navbar.Brand href="#top">EventHub</Navbar.Brand>
        <Navbar.Toggle aria-controls="eventhub-nav" />
        <Navbar.Collapse id="eventhub-nav">
          <Nav className="ms-auto">
            <Nav.Link href="#top">Home</Nav.Link>
            <Nav.Link href="#events">Events</Nav.Link>
            <Nav.Link href="#about">About</Nav.Link>
          </Nav>
        </Navbar.Collapse>
      </Container>
    </Navbar>
  );
}

```

```
}

export default AppNavbar;
```

VERIFY: Import tạm vào App.jsx và kiểm tra Brand/links/toggle không crash.

Không nhầm Nav.Link ở đây chưa phải React Router navigation. Slot 06 mới học Router/Routes/Link/NavLink.

CHECKPOINT — ☐ Navbar hiển thị | ☐ Responsive toggle không lỗi

Bước 14. Tạo HeroSection

THAO TÁC: Tạo src/components/HeroSection.jsx.

```
import { Container } from "react-bootstrap";

function HeroSection() {
  return (
    <section id="top" className="py-5 bg-light border-bottom">
      <Container>
        <h1 className="display-6 fw-bold">Discover Campus Events</h1>
        <p className="lead mb-0">
          Explore technology, career, community, sports and cultural activities.
        </p>
      </Container>
    </section>
  );
}

export default HeroSection;
```

CHECKPOINT — ☐ Heading hiển thị | ☐ Không import error

Bước 15. Tạo EventCard baseline với một record

THAO TÁC: Tạo src/components/EventCard.jsx. Trước tiên chỉ render một event prop, chưa Modal.

```
import { Badge, Card } from "react-bootstrap";

function EventCard({ event }) {
  return (
    <Card className="h-100 shadow-sm">
      <Card.Img variant="top" src={event.image} alt={event.title} />
      <Card.Body>
        <div className="d-flex justify-content-between align-items-start gap-2">
          <Card.Title>{event.title}</Card.Title>
          {event.featured && <Badge bg="warning" text="dark">Featured</Badge>}
        </div>
        <Card.Text className="mb-1"><strong>Category:</strong> {event.category}</Card.Text>
        <Card.Text className="mb-1"><strong>Date:</strong> {event.date}</Card.Text>
        <Card.Text><strong>Location:</strong> {event.location}</Card.Text>
      </Card.Body>
    </Card>
  );
}
```

```

    );
  }

  export default EventCard;

```

VERIFY: Trong App hoặc EventList tạm, truyền events[0]. Chỉ đi tiếp khi 1 card render đúng.

CHECKPOINT — ☐ Ảnh PASS | ☐ Title PASS | ☐ Category/date/location PASS

Bước 16. Củng cố Props: truyền event object

THAO TẮC: Trace chính xác object từ parent vào child.

```

<EventCard event={events[0]} />
function EventCard({ event }) {
  return <h2>{event.title}</h2>;
}

```

EVIDENCE: Ghi: Source = events[0]; Prop name = event; Receiver = EventCard; Rendered field = event.title.

CHECKPOINT — ☐ Giải thích được parent → prop → child | ☐ Không sửa trực tiếp event prop

Bước 17. Tạo EventList và render bằng map()

THAO TẮC: Tạo src/components/EventList.jsx với baseline không filter trước.

```

import { Col, Container, Row } from "react-bootstrap";
import { events } from "../data/events";
import EventCard from "../EventCard";

function EventList() {
  return (
    <Container id="events" className="py-5">
      <h2 className="mb-4">Upcoming Events</h2>
      <Row className="g-4">
        {events.map((event) => (
          <Col key={event.id} xs={12} md={6} lg={4}>
            <EventCard event={event} />
          </Col>
        ))}
      </Row>
    </Container>
  );
}

export default EventList;

```

WHY: map() là JavaScript Array method; mỗi record tạo ra một subtree UI. Nó không phải React Hook.

CHECKPOINT — ☐ 8 cards hiển thị | ☐ Không key warning | ☐ Các title khác nhau

Bước 18. Kiểm tra stable key

THAO TẮC: Xác nhận key={event.id}, không dùng Math.random().

WHY: Stable domain ID giúp React nhận diện các phần tử trong collection. Nếu child cần id, vẫn dùng event.id qua prop; key không phải normal prop.

CHECKPOINT — ☐ id unique | ☐ key dùng id

Bước 19. Kiểm tra responsive grid

THAO TÁC: Resize browser ở khoảng desktop/tablet/mobile.

EXPECTED: xs=12: 1 card/row; md=6: 2 cards/row; lg=4: 3 cards/row.

CHECKPOINT — ☐ Desktop hợp lý | ☐ Mobile không tràn ngang

Bước 20. Thêm search state và derived filtering

THAO TÁC: Trong EventList, import useState và tạo searchText.

```
import { useState } from "react";

const [searchText, setSearchText] = useState("");
```

THAO TÁC: Thêm Form.Control phía trên Row.

```
<Form.Control
  type="search"
  placeholder="Search events..."
  value={searchText}
  onChange={(e) => setSearchText(e.target.value)}
/>
```

THAO TÁC: Tạo derived list.

```
const filteredEvents = events.filter((event) =>
  event.title.toLowerCase().includes(searchText.trim().toLowerCase())
);
```

Đừng quên import Form từ react-bootstrap và đổi events.map thành filteredEvents.map.

CHECKPOINT — ☐ Gõ AI → thấy workshop nếu phù hợp | ☐ Xóa search → đủ 8 records

Bước 21. Thêm category filter

THAO TÁC: Tạo category state và list category.

```
const [category, setCategory] = useState("All");
const categories = ["All", ...new Set(events.map((event) => event.category))];
```

THAO TÁC: Thêm Form.Select và mở rộng filter.

```
<Form.Select value={category} onChange={(e) => setCategory(e.target.value)}>
  {categories.map((item) => (
    <option key={item} value={item}>{item}</option>
  ))}
</Form.Select>

const filteredEvents = events.filter((event) => {
  const matchesText = event.title
    .toLowerCase()
    .includes(searchText.trim().toLowerCase());
  const matchesCategory = category === "All" || event.category === category;
  return matchesText && matchesCategory;
});
```

CHECKPOINT — ☐ All = 8 | ☐ Technology = đúng các event Technology | ☐ Không mất search behavior

Bước 22. Thêm Featured-only state

THAO TÁC: Tạo state boolean và Form.Check.

```
const [featuredOnly, setFeaturedOnly] = useState(false);
```

```

<Form.Check
  type="switch"
  id="featured-only"
  label="Featured only"
  checked={featuredOnly}
  onChange={(e) => setFeaturedOnly(e.target.checked)}
/>

```

THAO TÁC: Thêm điều kiện vào filter.

```

const matchesFeatured = !featuredOnly || event.featured;
return matchesText && matchesCategory && matchesFeatured;

```

CHECKPOINT — ☐ OFF = không lọc featured | ☐ ON = chỉ featured | ☐ Kết hợp search/category vẫn đúng

Bước 23. Thêm result count và empty state

THAO TÁC: Hiển thị derived count và condition khi danh sách rỗng.

```

<p className="text-muted mt-3 mb-0">
  Showing {filteredEvents.length} of {events.length} events
</p>
{filteredEvents.length === 0 ? (
  <div className="text-center py-5">
    <h4>No events found</h4>
    <p className="text-muted">Try changing your search or filters.</p>
  </div>
) : (
  <Row className="g-4">{/* map here */}</Row>
)}

```

WHY: filteredEvents.length là derived value; không cần một useState riêng cho count.

CHECKPOINT — ☐ Count đúng | ☐ Empty state xuất hiện với query không match

Bước 24. Thêm Modal detail bằng local useState trong EventCard

THAO TÁC: Cập nhật EventCard imports và local state.

```

import { useState } from "react";
import { Badge, Button, Card, Modal } from "react-bootstrap";

function EventCard({ event }) {
  const [showDetail, setShowDetail] = useState(false);

```

THAO TÁC: Thêm button vào Card.Body.

```

<Button variant="primary" onClick={() => setShowDetail(true)}>
  View Detail
</Button>

```

THAO TÁC: Thêm Modal ngay sau Card trong Fragment.

```

<Modal show={showDetail} onHide={() => setShowDetail(false)} centered>
  <Modal.Header closeButton>
    <Modal.Title>{event.title}</Modal.Title>
  </Modal.Header>
  <Modal.Body>
    <p><strong>Category:</strong> {event.category}</p>

```

```

    <p><strong>Date:</strong> {event.date}</p>
    <p><strong>Location:</strong> {event.location}</p>
    <p><strong>Seats:</strong> {event.seats}</p>
    <p className="mb-0">{event.description}</p>
  </Modal.Body>
  <Modal.Footer>
    <Button variant="secondary" onClick={() => setShowDetail(false)}>
      Close
    </Button>
  </Modal.Footer>
</Modal>

```

Vì Card và Modal là hai sibling JSX nodes, bao chúng trong Fragment `<>...</>`.

PREDICT BEFORE RUN: Ban đầu `showDetail=false`. Click View Detail → setter nhận `true` → component re-render → Modal `show=true`. Close → `false`.

CHECKPOINT — ☐ Open PASS | ☐ Correct event detail PASS | ☐ Close PASS

Bước 25. Phân tích Props vs State và ownership

Value	Loại	Owner/Source	Lý do
event	Prop	EventList/data module	Child nhận data, không sở hữu
searchText	State	EventList	User có thể thay đổi
category	State	EventList	Filter thay đổi theo input
featuredOnly	State	EventList	Toggle thay đổi
filteredEvents	Derived value	Tính từ data + state	Không cần duplicate state
showDetail	Local State	EventCard	Mỗi card tự kiểm soát Modal

Context decision Core project không cần Context: data flow chỉ từ EventList → EventCard và local filter state nằm đúng owner. Ghi “Context not needed yet” là một quyết định thiết kế hợp lệ.

Bước 26. Bug injection và human-first debugging

Thực hiện từng bug riêng lẻ, chẩn đoán rồi hoàn tác trước bug tiếp theo.

Bug injection	Triệu chứng	Evidence cần xem	Cách tìm
event.title → event.titel	Một title trống/undefined	Browser/JSX	Trace field name từ events.js
Xóa Bootstrap CSS import	UI mất Bootstrap style	Browser vẫn render, console sạch	Kiểm tra main.jsx import
onClick={setShowDetail(true)}	Render loop/behavior bất thường	Terminal/console	Event handler đang gọi ngay khi render
image="public/images/..."	Ảnh 404	Network/browser	public URL không chứa public
Bỏ key	React warning	Console	Kiểm tra map() children

CHECKPOINT — ☐ Có evidence cho ít nhất 3 bugs | ☐ Sửa một nguyên nhân/lần

Bước 27. Chạy manual test matrix

ID	Test	Expected
T01	npm run dev	Dev server starts
T02	Open dev URL	Page loads
T03	Navbar	Brand + links visible
T04	Default event list	8 cards
T05	Images	No broken image
T06	Search "AI"	Relevant title only
T07	Search case-insensitive	Same logical result
T08	Category Technology	Only Technology
T09	Category All	All other active conditions preserved
T10	Featured ON	Only featured=true
T11	Combine search + category	Intersection correct
T12	No-match query	Empty state visible
T13	Result count	Matches rendered cards
T14	View Detail event 1	Correct modal content
T15	Close Modal	Modal hidden
T16	Open different card	Correct different detail
T17	Mobile width	No horizontal overflow
T18	Browser console	No relevant error/warning
T19	Terminal	No compile/runtime blocking error
T20	Refresh	Static data app reloads successfully

Bước 28. Tạo Git checkpoint

THAO TÁC: Nếu repo chưa init: git init. Sau khi test chính PASS:

```
git status
git add .
git commit -m "Build integrated EventHub React application"
git log -1
```

EXPECTED: git log -1 hiển thị commit mới và working tree không còn file cần commit (trừ file chủ động bỏ qua).

CHECKPOINT — ☐ Commit tồn tại | ☐ Message có nghĩa | ☐ node_modules không bị commit

Bước 29. Production build và preview

THAO TÁC: Dừng dev server nếu cần rồi chạy:

```
npm run build
npm run preview
```

EXPECTED: Build tạo dist/ và preview server mở được ứng dụng production-like local build.

IF FAILED: Đọc build error; production build có thể phát hiện vấn đề mà dev server chưa làm bạn chú ý.

CHECKPOINT — ☐ Build PASS | ☐ Preview PASS

Bước 30. Final Verification Gate và Evidence Package

Evidence	Yêu cầu
E1 Component Tree	Ảnh hoặc text tree có trách nhiệm component
E2 Application	Screenshot default list
E3 Filters	Screenshot search/filter result
E4 Detail State	Screenshot Modal của một event
E5 Props Trace	events.js → EventList → EventCard → JSX
E6 State Trace	event → setter → new state → UI
E7 Console	Không blocking error
E8 Terminal	Dev/build output PASS
E9 Tests	Manual test matrix đã đánh dấu
E10 Git	git log -1 / commit hash

FINAL GATE — ☐ Core functions PASS | ☐ Evidence đủ | ☐ Giải thích được không cần AI

9. Source code tham khảo hoàn chỉnh

Cách dùng Chỉ dùng phần này để đối chiếu sau khi đã tự làm từng bước. Nếu copy ngay từ đầu, bài tập mất mục tiêu “trace data / identify owner / verify”.

9.1 src/main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "bootstrap/dist/css/bootstrap.min.css";
import "./App.css";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

9.2 src/App.jsx

```
import AppNavbar from "../components/AppNavbar";
import HeroSection from "../components/HeroSection";
import EventList from "../components/EventList";
import AppFooter from "../components/AppFooter";

function App() {
  return (
    <>
      <AppNavbar />
      <HeroSection />
      <EventList />
      <AppFooter />
    </>
  );
}

export default App;
```

9.3 src/components/EventList.jsx

```
import { useState } from "react";
import { Col, Container, Form, Row } from "react-bootstrap";
import { events } from "../data/events";
import EventCard from "../EventCard";

function EventList() {
  const [searchText, setSearchText] = useState("");
  const [category, setCategory] = useState("All");
  const [featuredOnly, setFeaturedOnly] = useState(false);

  const categories = ["All", ...new Set(events.map((event) => event.category))];

  const filteredEvents = events.filter((event) => {
    const matchesText = event.title
      .toLowerCase()
      .includes(searchText.trim().toLowerCase());
    const matchesCategory = category === "All" || event.category === category;
    const matchesFeatured = !featuredOnly || event.featured;
    return matchesText && matchesCategory && matchesFeatured;
  });

  return (
    <Container id="events" className="py-5">
      <div className="d-flex flex-column flex-lg-row justify-content-between gap-3 mb-4">
        <div>
          <h2 className="mb-1">Upcoming Events</h2>
          <p className="text-muted mb-0">

```

```

    Showing {filteredEvents.length} of {events.length} events

  </p>
</div>

<div className="d-flex flex-column flex-md-row gap-2">
  <Form.Control
    type="search"
    placeholder="Search events..."
    value={searchText}
    onChange={(e) => setSearchText(e.target.value)}
  />

  <Form.Select
    value={category}
    onChange={(e) => setCategory(e.target.value)}
  >
    {categories.map((item) => (
      <option key={item} value={item}>{item}</option>
    ))}
  </Form.Select>

  <Form.Check
    type="switch"
    id="featured-only"
    label="Featured only"
    checked={featuredOnly}
    onChange={(e) => setFeaturedOnly(e.target.checked)}
  />
</div>
</div>

{filteredEvents.length === 0 ? (
  <div className="text-center py-5 border rounded bg-light">
    <h4>No events found</h4>
    <p className="text-muted mb-0">Try changing your search or filters.</p>
  </div>
) : (
  <Row className="g-4">
    {filteredEvents.map((event) => (
      <Col key={event.id} xs={12} md={6} lg={4}>
        <EventCard event={event} />
      </Col>
    ))}
  </Row>
)}
</Container>
);

```

```
}

export default EventList;
```

9.4 src/components/EventCard.jsx

```
import { useState } from "react";
import { Badge, Button, Card, Modal } from "react-bootstrap";

function EventCard({ event }) {
  const [showDetail, setShowDetail] = useState(false);

  return (
    <>
      <Card className="h-100 shadow-sm">
        <Card.Img className="event-image" variant="top" src={event.image} alt={event.title} />
        <Card.Body className="d-flex flex-column">
          <div className="d-flex justify-content-between align-items-start gap-2">
            <Card.Title>{event.title}</Card.Title>
            {event.featured && <Badge bg="warning" text="dark">Featured</Badge>}
          </div>
          <Card.Text className="mb-1"><strong>Category:</strong> {event.category}</Card.Text>
          <Card.Text className="mb-1"><strong>Date:</strong> {event.date}</Card.Text>
          <Card.Text className="mb-3"><strong>Location:</strong> {event.location}</Card.Text>
          <Button className="mt-auto" onClick={() => setShowDetail(true)}>
            View Detail
          </Button>
        </Card.Body>
      </Card>

      <Modal show={showDetail} onHide={() => setShowDetail(false)} centered>
        <Modal.Header closeButton>
          <Modal.Title>{event.title}</Modal.Title>
        </Modal.Header>
        <Modal.Body>
          <p><strong>Category:</strong> {event.category}</p>
          <p><strong>Date:</strong> {event.date}</p>
          <p><strong>Location:</strong> {event.location}</p>
          <p><strong>Seats:</strong> {event.seats}</p>
          <p className="mb-0">{event.description}</p>
        </Modal.Body>
        <Modal.Footer>
          <Button variant="secondary" onClick={() => setShowDetail(false)}>
            Close
          </Button>
        </Modal.Footer>
      </Modal>
    </>
  );
}
```

```

    );
  }

  export default EventCard;

```

9.5 src/components/AppFooter.jsx

```

import { Container } from "react-bootstrap";

function AppFooter() {
  return (
    <footer id="about" className="border-top py-4 bg-light">
      <Container>
        <p className="mb-0 text-muted">SBA301 EventHub - Slot 05 reinforcement project</p>
      </Container>
    </footer>
  );
}

export default AppFooter;

```

9.6 src/App.css

```

.event-image {
  height: 180px;
  object-fit: cover;
}

body {
  min-width: 320px;
}

```

10. Debugging decision tree

```

Application fails
↓
Does npm run dev start?
↓ yes
Does browser reach Vite URL?
↓ yes
Any terminal compile error?
↓ no
Any browser console error?
↓ no
Data correct in events.js?
↓ yes
map() returns elements?
↓ yes
Prop name/destructuring correct?
↓ yes

```

Event handler fires?

↓ yes

State setter changes value?

↓ yes

JSX/Modal depends on that state?

↓ yes

Then inspect styling / edge cases

Symptom	Likely cause	First diagnostic
npm script not found	Wrong directory / package.json	Get-Location + check package.json
Module not found: react-bootstrap	Package missing	npm ls react-bootstrap
Bootstrap components unstyled	CSS import missing	Check main.jsx
Image broken	Wrong public URL/file name	Open image URL directly
Card title undefined	Field mismatch	Trace events.js → prop → JSX
No cards	map/filter returns none	console.log length temporarily
Key warning	Missing/stable key issue	Check key={event.id}
Modal opens immediately	Setter called during render	Check onClick
Modal does not open	Event/setter/state binding	Trace click → set → show
Filter inconsistent	Boolean/string logic	Evaluate each matches* value

11. Human Verification Gate – không dùng AI

1. Vì sao events.js không phải database?
2. map() là gì và vì sao không phải Hook?
3. Tại sao dùng key={event.id}?
4. event là Props hay State? Ai sở hữu nó?
5. Tại sao child không nên sửa event.title trực tiếp?
6. searchText là Props hay State? Tại sao?
7. filteredEvents có cần useState không? Vì sao?
8. Mô tả event click → setShowDetail → re-render → Modal.
9. Tại sao Context không bắt buộc trong project này?
10. Bootstrap CSS import khác npm install react-bootstrap bootstrap ở điểm nào?
11. Terminal và Browser Console cung cấp hai loại evidence khác nhau như thế nào?
12. Nếu search không ra kết quả, làm sao phân biệt bug với empty state đúng?
13. Tại sao cần làm một EventCard hoạt động trước khi map tất cả records?
14. Production build khác dev server ở mục đích kiểm tra nào?
15. Slot 06 sẽ giải quyết giới hạn nào của Nav.Link hiện tại?

Pass rule Sinh viên phải giải thích được ít nhất 12/15 câu bằng lời của mình và trace được một record + một state transition. “App chạy” nhưng không giải thích được data flow chưa được xem là hoàn thành năng lực.

12. AI-augmented workflow cho project

AI chỉ được dùng sau khi có evidence. Trước mỗi lần hỏi AI, sinh viên phải ghi Expected, Actual, first error, relevant file/code và last change.

12.1 Prompt trace Props

```
Act as a React tutor. Do not rewrite my files.
Trace exactly:
static event object → EventList → map callback → event prop → EventCard destructuring → JSX.
Find the FIRST mismatch only.
Data: [PASTE]
Parent: [PASTE]
Child: [PASTE]
Actual result: [PASTE]
```

12.2 Prompt trace State

```
My EventHub detail Modal does not update correctly.
Expected: [EXPECTED]
Actual: [ACTUAL]
State: [STATE CODE]
Button/handler: [CODE]
Modal: [CODE]
Console: [ERROR]
Do not rewrite the component.
Trace click → handler → setter → new state → rendered Modal and identify the first broken stage.
```

12.3 Prompt architecture review

```
Review this SBA301 Slot 05 frontend design.
Do NOT add Router, Axios, backend, Redux, Zustand, React Query or TypeScript.
Check only:
1) component responsibilities;
2) props flow;
3) state ownership;
4) duplicated state;
5) unnecessary Context;
6) map/key usage.
Explain problems before suggesting changes.
```

AI hallucination check Sai: “useState tự lưu filter vào SQL Server”. Đúng: useState là client-side component state; database persistence chỉ xuất hiện sau này qua HTTP/REST → Spring Boot → persistence/database.

13. Project-Based Learning Transfer

Sau khi hoàn thành EventHub, mỗi nhóm chuyển cùng pattern sang đề tài chính. Không copy domain EventHub; chỉ tái sử dụng cách tư duy.

EventHub Pattern	Ví dụ News	Ví dụ Hotel
------------------	------------	-------------

EventHub Pattern	Ví dụ News	Ví dụ Hotel
events.js	newsData.js	roomsData.js
EventList	NewsList	RoomList
EventCard	NewsCard	RoomCard
event prop	article prop	room prop
showDetail state	expanded state	showDetail state
search/category filters	search/category	search/type/availability

Mini-deliverable nhóm sau Slot 05: ít nhất 5 sample records, một reusable item component, map()+stable key, một local state interaction, React-Bootstrap layout, không blocking errors và một Git commit.

14. Worksheets và bảng kiểm nộp bài

14.1 Props Trace Worksheet

Mục	Sinh viên điền
Value/record	
Defined in	
Imported/used by	
Prop name	
Received by	
Rendered field	
Evidence	

14.2 State Trace Worksheet

Mục	Sinh viên điền
State name	
Owner component	
Initial value	
User event	
Updater	
New value	

Mục	Sinh viên điền
UI change	
Evidence	

14.3 Final Verification Matrix

Requirement	Evidence	Result
Dev server	Terminal	PASS / FAIL
Default list	Browser	PASS / FAIL
8 records	Data vs UI	PASS / FAIL
Props	Code trace	PASS / FAIL
map/key	Code + Console	PASS / FAIL
Search	Interaction	PASS / FAIL
Category	Interaction	PASS / FAIL
Featured filter	Interaction	PASS / FAIL
Empty state	Interaction	PASS / FAIL
Modal open/close	Interaction	PASS / FAIL
Responsive grid	Browser resize	PASS / FAIL
Console	DevTools	PASS / FAIL
Production build	Terminal	PASS / FAIL
Git	git log -1	PASS / FAIL

15. Tự đánh giá chất lượng

Tiêu chí	0	1	2	3
Functional correctness	Không chạy	Chạy một phần	Core chạy	Core + edge cases PASS
Component structure	Monolithic	Tách chưa rõ	Trách nhiệm khá rõ	Tree hợp lý, giải thích tốt
Props/data flow	Không hiểu	Có prop nhưng sai	Prop đúng	Trace end-to-end
State/interaction	Không có	State lỗi	Core interaction đúng	Ownership/derived data đúng
Verification	Không evidence	Screenshot đơn lẻ	Tests + console	Evidence có hệ thống
Explanation	Không giải thích	Nói cú pháp	Giải thích flow	Giải thích + justify design

16. Bài mở rộng – chỉ thực hiện sau khi Core PASS

- Extension A: thêm Reset Filters button.
- Extension B: nâng state lên EventList để dùng một Modal duy nhất với selectedEvent (stronger-student challenge).
- Extension C: thêm Favorite state local; giải thích vì sao reload sẽ mất favorite.
- Extension D: thử ThemeContext cho Navbar/Footer và viết 3 câu đánh giá Context có thực sự cần không.
- Extension E: thêm sort theo date hoặc title bằng derived data, không duplicate source array.
- Extension F: thêm aria-label phù hợp cho control và kiểm tra keyboard navigation cơ bản.

Không vượt scope Không thêm Router/Axios/backend chỉ để project “trông nâng cao”. Mục tiêu Slot 05 là tích hợp và chứng minh chắc Component + Props + State + Collection Rendering + Verification.

17. Quick Command Sheet

```
# Environment
node -v
npm -v
git --version

# Project setup
npm create vite@5 eventhub-campus-explorer -- --template react
cd eventhub-campus-explorer
npm install
npm install react-bootstrap bootstrap

# Development
npm run dev

# Verification/build
npm run build
npm run preview

# Git checkpoint
git status
git add .
git commit -m "Build integrated EventHub React application"
git log -1
```

18. Cầu nối sang Slot 06

EventHub hiện có các mục Home | Events | About nhưng chúng chưa phải các route SPA thật. Toàn bộ ứng dụng vẫn chủ yếu là một screen, dùng anchor/section. Câu hỏi tiếp theo là: làm thế nào mỗi view có URL riêng, chuyển view mà không reload toàn trang, và truyền id qua URL?

```
Slot 05
Collection + Props + State + Modal
↓
```

Slot 06

SPA Navigation

React Router

Routes / Route

Link / NavLink

URL Parameters

19. Final Student Checklist

- Project chạy bằng IntelliJ npm Run Configuration hoặc npm run dev.
- Có 8 event records và render đúng số card.
- Một EventCard được tái sử dụng cho nhiều record.
- Dùng map() và stable key.
- Search/category/featured state hoạt động kết hợp.
- Không lưu derived filteredEvents vào state không cần thiết.
- Modal detail mở/đóng bằng local state.
- Không sửa Props trong child.
- Không dùng Context nếu không có lý do.
- Không dùng Router/Axios/backend trong core Slot 05.
- Browser console và terminal không có blocking error.
- Manual test matrix đã thực hiện.
- npm run build PASS.
- Git commit tồn tại.
- Có Props Trace + State Trace + Component Tree.
- Giải thích được app không dùng AI.

20. Căn cứ nội dung

Tài liệu này là project củng cố cho SBA301-40 v2 – Slot 05 Integrated React Lab 01. Nó giữ các năng lực cốt lõi của Lab 01 và kiến thức tổng hợp Chapter 01–05: Vite, component architecture, React-Bootstrap, static collection data, Props, State/useState, events, map/key, debugging, verification và Git. Modal/detail được dùng như nội dung consolidation/bridge từ Lab 02; không được ghi nhận như yêu cầu nguyên bản của Lab 01.

EventHub là domain transfer do tài liệu hướng dẫn này xây dựng để sinh viên chứng minh khả năng áp dụng kiến thức sang bài toán khác, không phải official Lab 01 domain.